



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Testprotokolle

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 9. Juli 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Mohammed Alhalabi	fertig
1.1	Mohammed Alhalabi	fertig
1.2	Mohammed Alhalabi	fertig
1.3	Mohammed Alhalabi	fertig
2	Mohammad Khatib	fertig
2.1	Mohammad Khatib	fertig
2.2	Mohammad Khatib	fertig
2.3	Mohammad Khatib	fertig
3	Mohammed Alhalabi	fertig
3.1	Mohammed Alhalabi	fertig
3.2	Mohammed Alhalabi	fertig
3.3	Mohammed Alhalabi	fertig
4	Mohamed Sabri Jlizi	fertig
4.1	Mohamed Sabri Jlizi	fertig
4.2	Mohamed Sabri Jlizi	fertig
4.3	Mohamed Sabri Jlizi	fertig
5	Lucas Neidahl	fertig
5.1	Lucas Neidahl	fertig
5.2	Lucas Neidahl	fertig
5.3	Lucas Neidahl	fertig
6	Lucas Neidahl	fertig
6.1	Lucas Neidahl	fertig
6.2	Lucas Neidahl	fertig
6.3	Lucas Neidahl	fertig
7	Mohamed Sabri Jlizi	fertig
7.1	Mohamed Sabri Jlizi	fertig
7.2	Mohamed Sabri Jlizi	fertig
7.3	Mohamed Sabri Jlizi	fertig
8	Mohamed Sabri Jlizi , Mohammad Khatib	fertig
8.1	Mohamed Sabri Jlizi , Mohammad Khatib	fertig
8.2	Mohamed Sabri Jlizi , Mohammad Khatib	fertig

8.3	Mohamed Sabri Jlizi , Mohammad Khatib	fertig
9	Luan Hyseni	fertig
9.1	Luan Hyseni	fertig
9.2	Luan Hyseni	fertig
9.3	Luan Hyseni	fertig
10	Luan Hyseni	fertig
10.1	Luan Hyseni	fertig
10.2	Luan Hyseni	fertig
10.3	Luan Hyseni	fertig
11	Luan Hyseni	fertig
11.1	Luan Hyseni	fertig
11.2	Luan Hyseni	fertig
11.3	Luan Hyseni	fertig
12	Luan Hyseni	fertig
12.1	Luan Hyseni	fertig
12.2	Luan Hyseni	fertig
12.3	Luan Hyseni	fertig
13	Lucas Neidahl	fertig
13.1	Luan Hyseni	fertig
13.2	Luan Hyseni	fertig
13.3	Luan Hyseni	fertig
14.1	Lucas Neidahl	fertig
14.2	Lucas Neidahl	fertig
14.3	Lucas Neidahl	fertig
15	Lucas Neidahl	fertig
15.1	Lucas Neidahl	fertig
15.2	Lucas Neidahl	fertig
15.3	Lucas Neidahl	fertig
16	Marvin Voigt	fertig
16.1	Marvin Voigt	fertig
16.2	Marvin Voigt	fertig
16.3	Marvin Voigt	fertig
17	Lucas Neidahl, Marvin Voigt	fertig
17.1	Lucas Neidahl , Marvin Voigt	fertig

SEP2025

Drone Choreo "SkyOrchestrator"

17.2	Lucas Neidahl , Marvin Voigt	fertig
17.3	Lucas Neidahl , Marvin Voigt	fertig
18	Lucas Neidahl	fertig
18.1	Lucas Neidahl	fertig
18.2	Lucas Neidahl	fertig
18.3	Lucas Neidahl	fertig

Inhaltsverzeichnis

1	Testdurchführung (2025-07-06)	8
1.1	Testumgebung	8
1.2	Testprotokoll	8
1.3	Zusammenfassung	9
2	Testdurchführung (2025-07-03)	10
2.1	Testumgebung	10
2.2	Testprotokoll	10
2.3	Zusammenfassung	11
3	Testdurchführung (2025-07-06)	12
3.1	Testumgebung	12
3.2	Testprotokoll	12
3.3	Zusammenfassung	13
4	Testdurchführung (2025-07-03)	14
4.1	Testumgebung	14
4.2	Testprotokoll	14
4.3	Zusammenfassung	15
5	Integrationstests (2025-07-04)	16
5.1	Testumgebung	16
5.2	Testprotokoll	16
5.3	Zusammenfassung	17
6	Integrationstests (2025-07-05)	18
6.1	Testprotokoll	18
6.2	Zusammenfassung	19
7	Testdurchführung (2025-07-07)	20
7.1	Testumgebung	20
7.2	Testprotokoll	20
7.3	Zusammenfassung	21

8 Testdurchführung (2025-07-03)	22
8.1 Testumgebung	22
8.2 Testprotokoll	22
8.3 Zusammenfassung	23
9 Integrationstests (2025-07-09)	24
9.1 Testumgebung	24
9.2 Testprotokoll	24
9.3 Zusammenfassung	25
10 Integrationstests (2025-07-09)	26
10.1 Testumgebung	26
10.2 Testprotokoll	26
10.3 Zusammenfassung	27
11 Integrationstests (2025-07-09)	28
11.1 Testumgebung	28
11.2 Testprotokoll	28
11.3 Zusammenfassung	29
12 Integrationstests (2025-07-09)	30
12.1 Testumgebung	30
12.2 Testprotokoll	30
12.3 Zusammenfassung	31
13 Integrationstests (2025-07-09)	32
13.1 Testumgebung	32
13.2 Testprotokoll	32
13.3 Zusammenfassung	33
14 Komponententests (2025-07-05)	34
14.1 Testumgebung	34
14.2 Testprotokoll	34
14.3 Zusammenfassung	35
15 Komponententests (2025-07-05)	36
15.1 Testumgebung	36
15.2 Testprotokoll	36
15.3 Zusammenfassung	37
16 Testdurchführung (2025-07-08)	38
16.1 Testumgebung	38

16.2 Testprotokoll	38
16.3 Zusammenfassung	39
17 Testdurchführung (2025-07-08)	40
17.1 Testumgebung	40
17.2 Testprotokoll	40
17.3 Zusammenfassung	41
18 Testdurchführung (2025-07-08)	42
18.1 Testumgebung	42
18.2 Testprotokoll	42
18.3 Zusammenfassung	43

1 Testdurchführung (2025-07-06)

Art des Tests: Abnahmetest

Ausgeführte Testfälle: **T100**

Beteiligte Tester: Mohammed Alhalabi

Abgedeckte Funktionen: **F10, F11, F12, F13, F14**

1.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 Home (64 Bit), Version 24H2, Intel Core i7-11800H (11. Gen), 16 GB RAM ,NVIDIA GeForce RTX 3060 Laptop GPU, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 06.07.2025)

1.2 Testprotokoll

Testfall	<i>T100 – Szene erstellen und speichern</i>
Tester	<i>Mohammed Alhalabi</i>
Eingaben	Projektname: Kreis Anzahl Drohnen: 25 Drohnenmodell: quadrotor Hintergrund: standard Start der Simulation über Button „Simulation starten“
Soll - Reaktion	Die Szene wird korrekt in der Szenenliste angezeigt, alle eingegebenen Metadaten (Name, Dauer, Formation, Lichteffekte) sind übernommen, es erscheint keine Fehlermeldung, in der Logdatei steht: „Szene erfolgreich gespeichert.“

Ist – Reaktion	Die Szene Sonnenaufgang wurde erfolgreich gespeichert. Sie erscheint vollständig in der Liste. Keine Fehlermeldung in GUI oder Konsole. Logmeldung vorhanden.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Nicht notwendig

1.3 Zusammenfassung

Der Testfall **T100** wurde erfolgreich durchgeführt. Die Erstellung und Speicherung einer neuen Szene funktionierte vollständig und fehlerfrei. Die Benutzereingaben wurden korrekt übernommen und in der GUI visualisiert. Das Systemverhalten entsprach den Erwartungen. Eine Abnahme der Funktion „Szene erstellen und speichern“ ist somit gerechtfertigt.

2 Testdurchführung (2025-07-03)

Art des Tests: Abnahmetest

Ausgeführte Testfälle: **T200**

Beteiligte Tester: Mohammad Khatib

Abgedeckte Funktionen: **F20**

2.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 (64 Bit), Intel Core i7-7700K , 32 GB RAM, NVIDIA GeForce RTX 2080, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 03.07.2025)

2.2 Testprotokoll

Testfall	<i>T200 – Simulation einer Drohnenshow</i>
Tester	<i>Mohammad Khatib</i>
Eingaben	Formation: „Eigener Text/Formation“ Anzahl Drohnen: 50 Anzahl Textwechsel: 1 Text 1: HELLO Zeitpunkt: 0.0 Sekunden Drohnenmodell: quadrotor Hintergrund: #standard Start der Simulation über Button „Simulation starten“
Soll - Reaktion	ohne Fehlermeldung starten, alle 50 Drohnen in die Formation „HELLO“ überführen, die Animation flüssig und synchron anzeigen, visuelle Drohnenbewegung sowie Lichteffekte korrekt darstellen, in der Logdatei eine Erfolgsmeldung schreiben (z. B. „Simulation erfolgreich ausgeführt“).

Ist – Reaktion	Die Simulation wurde erfolgreich gestartet. Die Drohnen bewegten sich korrekt in die Formationspositionen des Wortes „HELLO“. Die Animation lief flüssig in der PyBullet-Ansicht. Die GUI blieb stabil und reagierte korrekt. Kein Fehler in der Konsole.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Nicht notwendig

2.3 Zusammenfassung

Der Testfall **T200** wurde erfolgreich durchgeführt. Die Anwendung erfüllte alle definierten Muss-Kriterien im Zusammenhang mit der 3D-Simulation (Funktion $\langle F20 \rangle$) vollständig. Die Show wurde korrekt visualisiert, die Drohnenbewegungen waren flüssig, und das System zeigte ein stabiles Verhalten. Eine Abnahme der Funktion „Simulation einer Drohnenshow“ ist somit gerechtfertigt.

3 Testdurchführung (2025-07-06)

Art des Tests: Abnahmetest

Ausgeführte Testfälle: **T300**

Beteiligte Tester: Mohammed Alhalabi

Abgedeckte Funktionen: **F30**

3.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 Home (64 Bit), Version 24H2, Intel Core i7-11800H (11. Gen), 16 GB RAM ,NVIDIA GeForce RTX 3060 Laptop GPU, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 06.07.2025)

3.2 Testprotokoll

Testfall	<i>T300 – Projekt speichern und laden</i>
Tester	<i>Mohammed Alhalabi</i>
Eingaben	Projektname: Herz Anzahl Drohnen: 20 Drohnenmodell: quadrotor Hintergrund: standard Start der Simulation über Button „Simulation starten“
Soll - Reaktion	Das Projekt wird vollständig gespeichert, nach dem Neustart kann das Projekt wieder geöffnet werden, alle Szenen erscheinen wie vor dem Schließen, die GUI ist fehlerfrei benutzbar.

Ist – Reaktion	Das Projekt <code>Testprojekt_01</code> wurde erfolgreich gespeichert und geladen. Die Szenen „Herz“ waren vollständig vorhanden. Alle Metadaten und Formationen wurden korrekt angezeigt. Keine Fehlermeldungen in GUI oder Konsole.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Nicht notwendig

3.3 Zusammenfassung

Der Testfall **T300** wurde erfolgreich durchgeführt. Die Speicher- und Ladefunktion ermöglichte die vollständige Wiederherstellung eines Projekts nach dem Neustart der Anwendung. Szenen, Formationen und Metadaten wurden korrekt geladen. Das Systemverhalten war stabil und entsprach den Erwartungen. Eine Abnahme der Funktion „Projekt verwalten“ ist damit gerechtfertigt.

4 Testdurchführung (2025-07-03)

Art des Tests: Abnahmetest

Ausgeführte Testfälle: **T400**

Beteiligte Tester: Mohamed Sabri Jlizi

Abgedeckte Funktionen: **F40**

4.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 (64 Bit), Intel Core i7-7700K , 32 GB RAM, NVIDIA GeForce RTX 2080, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 03.07.2025)

4.2 Testprotokoll

Testfall	<i>T400 – Automatische Erkennung von Eingabefehlern</i>
Testers	<i>Mohamed Sabri Jlizi</i>
Eingaben	Formationstyp: „Eigener Text/Formation“ Anzahl Textwechsel: 1 Text 1: leer gelassen Zeitpunkt 1: 0.0 Anzahl Drohnen: 100 Klick auf „Simulation starten“
Soll - Reaktion	Die Software erkennt den leeren Texteingabe-Feldinhalt und verhindert die Ausführung. Die folgende Fehlermeldung wird in der GUI angezeigt: "Fehler: Textfeld 1 darf nicht leer sein."
Ist – Reaktion	Die Fehlermeldung wurde wie erwartet angezeigt. Die Simulation wurde nicht gestartet. Die Eingabefelder blieben fokussiert. Kein Absturz oder Hängenbleiben des Programms.

Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Nicht notwendig

4.3 Zusammenfassung

Der Testfall **T400** wurde erfolgreich durchgeführt. Die Eingabvalidierung in der GUI funktioniert korrekt: Leere oder ungültige Texteingaben werden erkannt, der Start der Simulation wird verhindert und eine klare Fehlermeldung angezeigt. Die Anwendung reagierte stabil und nachvollziehbar. Das Verhalten erfüllt die Muss-Anforderung $\langle F40 \rangle$ und trägt zur Fehlervermeidung und Benutzerfreundlichkeit bei.

5 Integrationstests (2025-07-04)

Art des Tests: Integrationstest

Ausgeführte Testfälle: **T500**

Beteiligte Tester: Lucas Neidahl

Abgedeckte Komponenten: **<C10+C20>** (GUI + Show-Konfigurationsmodul)

5.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt.

5.2 Testprotokoll

Testfall	T500 – Datenaustausch zwischen GUI und Show-Konfigurator (Integrationstest)
Tester	Lucas Neidahl
Eingaben	Erstellung einer neuen Szene über die GUI: Szenenname: Integrationstest_Szene Anzahl Drohnen: 30 Formation: Kreis Klick auf den Button „Szene speichern“.
Soll - Reaktion	Die in der GUI (<C10>) eingegebenen Parameter werden korrekt an das Show-Konfigurationsmodul (<C20>) übergeben. Das Modul erstellt intern ein Szenenobjekt mit den exakten Werten, ohne Datenverlust oder -verfälschung. Als Rückmeldung erscheint die neue Szene mit dem korrekten Namen in der Szenenliste der GUI. Es treten keine Fehler in der Konsole auf.

Ist – Reaktion	Die Eingabedaten wurden fehlerfrei und vollständig von der GUI (<C10>) an das Konfigurationsmodul (<C20>) übergeben. Die interne Datenstruktur der neu erstellten Szene entsprach exakt den Eingaben. Die Szene Integrationstest_Szene wurde wie erwartet korrekt in der Liste der GUI angezeigt.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

5.3 Zusammenfassung

Der Integrationstest **T500** wurde erfolgreich durchgeführt. Die Kommunikation zwischen der Benutzeroberfläche (<C10>) und dem Show-Konfigurationsmodul (<C20>) funktioniert reibungslos. Nutzereingaben zum Anlegen einer Szene werden korrekt übermittelt, verarbeitet und das Ergebnis wird in der GUI wie erwartet visualisiert. Das Zusammenspiel beider Komponenten ist stabil und fehlerfrei.

6 Integrationstests (2025-07-05)

Art des Tests: Integrationstest

Ausgeführte Testfälle: **T530**

Beteiligte Tester: Lucas Neidahl

Abgedeckte Komponenten: <C10+C30> (GUI + Projektmanagement-Modul)

6.1 Testprotokoll

Testfall	T530 – Projekt speichern und laden über die GUI
Tester	Lucas Neidahl
Eingaben	1. In der GUI wird ein Projekt mit zwei Szenen erstellt: SzeneA, SzeneB 2. Auf den Menüpunkt oder Button 'Projekt speichern' in der GUI klicken. 3. Speichern der Datei unter dem Namen 'GUIIntegrations-test.json'. 4. Neustart der Anwendung. 5. Klick auf 'Projekt laden' und Auswahl der zuvor gespeicherten Datei.
Soll - Reaktion	Die Aktionen in der GUI (<C10>) rufen die korrekten Funktionen im Projektmanagement-Modul (<C30>) auf. Das Projekt wird fehlerfrei auf die Festplatte geschrieben. Nach dem Laden des Projekts sind beide Szenen SzeneA und SzeneB mit all ihren Einstellungen wieder vollständig und korrekt in der GUI sichtbar. Der gesamte Prozess verläuft ohne Abstürze oder Fehlermeldungen.

Ist – Reaktion	Die Interaktion zwischen der GUI und dem Projektmanagement-Modul funktionierte reibungslos. Das Projekt wurde erfolgreich gespeichert und nach dem Neustart der Anwendung vollständig wiederhergestellt. Alle Szenen und deren Konfigurationen wurden korrekt in der Benutzeroberfläche geladen und angezeigt.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Nicht notwendig

6.2 Zusammenfassung

Der Integrationstest **T530** wurde erfolgreich abgeschlossen. Das Zusammenspiel zwischen der Benutzeroberfläche (<C10>) und dem Backend für das Projektmanagement (<C30>) ist funktionsfähig und stabil. Der für den Benutzer zentrale Arbeitsablauf des Speicherns und Ladens eines gesamten Projekts funktioniert wie erwartet, wobei die Datenintegrität gewahrt bleibt.

7 Testdurchführung (2025-07-07)

Art des Tests: Integrationstest

Ausgeführte Testfälle: **T510**

Beteiligte Tester: Mohamed Sabri Jlizi

Abgedeckte Komponenten: <**C20**+**C60**> (ShowKonfigurator+ Simulation)

7.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 (64 Bit), Intel Core i7-7700K , 32 GB RAM, NVIDIA GeForce RTX 2080, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 07.07.2025)

7.2 Testprotokoll

Testfall	<i>T510 – Übergabe der Showkonfiguration an die Simulation</i>
Tester	<i>Mohamed Sabri Jlizi</i>
Eingaben	Der Benutzer öffnete das SkyOrchestrator-GUI und wählte als Formationstyp „Eigener Text/Formation“. In den Textwechsel-Eingabefeldern wurde das Wort „HELLO“ eingegeben, das nach 0.0 Sekunden erscheinen soll. Zusätzlich wurde eine Drohnenanzahl von 120 festgelegt und das Standard-Theme sowie das Drohnenmodell „quadrotor“ ausgewählt. Die Eingaben wurden anschließend über den Button „Simulation starten“ an die Simulation übergeben.“

Soll - Reaktion	Die vom Benutzer im GUI eingegebene Konfiguration soll vollständig und korrekt an das Simulation-Modul übergeben werden. Dazu gehört insbesondere die Übergabe des eingegebenen Textes, der zugehörigen Zeitangabe, der Gesamtanzahl der Drohnen sowie der Theme- und Modellparameter. Die Simulation soll diese Konfigurationsdaten übernehmen, korrekt interpretieren und entsprechend initialisieren. Fehlerhafte oder unvollständige Übergaben dürfen nicht auftreten.
Ist – Reaktion	Nach dem Betätigen des Start-Buttons wurde die Konfiguration erfolgreich übergeben. In der Konsole wurden die übergebenen Parameter korrekt angezeigt. Die Simulation startete daraufhin automatisch und zeigte die Drohneninformation des Wortes „HELLO“ in PyBullet. Die Eingaben zur Drohnenanzahl und zum Zeitpunkt wurden korrekt übernommen. Auch die gewählten Einstellungen für Hintergrund und Modell wurden sichtbar angewendet. Während des Ablaufs traten keine Fehler oder Warnmeldungen in der GUI oder Konsole auf.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

7.3 Zusammenfassung

Der Integrationstest **T510** wurde erfolgreich durchgeführt. Die Schnittstelle zwischen dem Show-Konfigurator (<C20>) und dem Simulationsmodul (<C60>) funktionierte erwartungsgemäß. Alle im GUI erfassten Parameter wurden korrekt übergeben und in der Simulation umgesetzt. Es konnten keine Übertragungsfehler festgestellt werden. Damit ist die Funktionsfähigkeit dieser Schnittstelle für den Anwendungsfall „Texteingabe + Zeitsteuerung“ erfolgreich validiert.

8 Testdurchführung (2025-07-03)

Art des Tests: Abnahmetest

Ausgeführte Testfälle: **T520**

Beteiligte Tester: Mohammad Khatib, Mohamed Sabri Jlizi

Abgedeckte Komponenten: <**C10+C60**> (GUI + Simulation)

8.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 11 (64 Bit), Intel Core i7-7700K , 32 GB RAM, NVIDIA GeForce RTX 2080, Python-Version 3.13, PyBullet, Tkinter, SkyOrchestrator (Stand 03.07.2025)

8.2 Testprotokoll

Testfall	<i>T520 – Start der Simulation aus der Benutzeroberfläche</i>
Tester	<i>Mohammad Khatib, Mohamed Sabri Jlizi</i>
Eingaben	Formation: „Eigener Text/Formation“ Anzahl Drohnen: 50 Anzahl Textwechsel: 1 Text 1: HELLO Zeitpunkt: 0.0 Sekunden Drohnenmodell: quadrotor Hintergrund: #standard Start der Simulation über Button „Simulation starten“

Soll - Reaktion	Die GUI sollte alle eingegebenen Parameter korrekt an die Simulationsfunktion <code>run_simulation()</code> übergeben. Nach dem Start sollte sich die Simulation in einem PyBullet-Fenster öffnen und die Szene darstellen. Die Drohnen sollten sich wie erwartet in die Formation des Wortes „HELLO“ bewegen. Zusätzlich sollte die Tastatursteuerung zur Kameranavigation funktionieren. Während des Ablaufs sollten weder im GUI noch in der Konsole Fehlermeldungen auftreten.
Ist – Reaktion	Die Simulation wurde wie erwartet gestartet, nachdem der Benutzer die Eingaben vorgenommen hatte. Alle Parameter wurden korrekt übergeben und das PyBullet-Fenster öffnete sich automatisch. Die Drohnen flogen flüssig in ihre Zielpositionen entsprechend der Formation „HELLO“. Die Steuerung der Kamera mit der Tastatur funktionierte ebenfalls wie vorgesehen. Während der gesamten Simulation traten keine Fehler oder unerwarteten Meldungen in der Konsole oder GUI auf.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

8.3 Zusammenfassung

Der Integrationstestfall **T520**, der die Übergabe der Benutzereingaben von der GUI an das Simulationsmodul prüft, wurde erfolgreich durchgeführt. Die Komponente <C10> (GUI) übergab sämtliche Parameter fehlerfrei an <C60> (Simulation), wodurch die Formation des Wortes „HELLO“ korrekt in der 3D-Umgebung dargestellt wurde. Die Simulation konnte ohne Unterbrechung gestartet und visuell verfolgt werden. Die erwarteten Anforderungen wurden erfüllt und das Zusammenspiel beider Komponenten ist damit als funktionsfähig und stabil zu bewerten.

9 Integrationstests (2025-07-09)

Art des Tests: Integrationstest

Ausgeführte Testfälle: T530

Beteiligte Tester: Luan Hyseni

Abgedeckte Komponenten: <C10 + C30> (GUI + Projektmanagement)

9.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendete Versionen: Python 3.13, Tkinter, PyBullet, SkyOrchestrator (Stand: 09.07.2025)

9.2 Testprotokoll

Testfall	T530 – Integrationstest: GUI und Projektmanagement
Tester	Luan Hyseni
Eingaben	1. Projektname: T530_TestSzene 2. Formation: Kreis 3. Drohnenanzahl: 20 4. Lichteffect: Blau 5. Projekt speichern, Anwendung schließen 6. Anwendung neu starten, Projekt laden
Soll - Reaktion	Projekt wird erfolgreich gespeichert Nach Neustart kann das Projekt korrekt geladen werden Alle Metadaten (Formation, Drohnenanzahl, Effekte) sind korrekt wiederhergestellt GUI bleibt stabil und ohne Fehlermeldung
Ist – Reaktion	Projekt wurde korrekt gespeichert und geladen Alle Inhalte waren vollständig Keine Fehler in GUI oder Konsole
Ergebnis	Bestanden

Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Keine erforderlich

9.3 Zusammenfassung

Der Integrationstest T530 wurde erfolgreich durchgeführt. Die Daten aus der GUI wurden korrekt über das Projektmanagement gespeichert und nach einem Neustart vollständig wiederhergestellt. Dies bestätigt die stabile Kommunikation der Komponenten C10 und C30.

10 Integrationstests (2025-07-09)

Art des Tests: Integrationstest

Ausgeführte Testfälle: T540

Beteiligte Tester: Luan Hyseni

Abgedeckte Komponenten: <C50 + C60> (Lichteffekte + Simulation)

10.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendete Versionen: Python 3.13, Tkinter, PyBullet, SkyOrchestrator (Stand: 09.07.2025)

10.2 Testprotokoll

Testfall	T540 – Integrationstest: Lichteffekte und Simulation
Tester	Luan Hyseni
Eingaben	1. Formation: Kreis 2. Drohnenanzahl: 10 3. Lichteffekt: Blau (konstant) 4. Szene mit Lichteffekt erstellen 5. Simulation starten
Soll - Reaktion	Der definierte Lichteffekt wird korrekt in der Simulation dargestellt Die betroffenen Drohnen zeigen die eingestellte Farbe Die Anzeige ist synchron zum Szenenstart Keine Fehlermeldungen oder Abstürze
Ist – Reaktion	Die Drohnen wurden korrekt in Kreisformation angezeigt Alle Drohnen zeigten das blaue Licht wie konfiguriert Keine Fehler im Terminal oder der GUI
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine

Nacharbeiten	Keine erforderlich
--------------	--------------------

10.3 Zusammenfassung

Der Integrationstest T540 wurde erfolgreich durchgeführt. Die definierten Lichteffekte wurden korrekt aus der Effektdatenbank übernommen und in der Simulation sichtbar dargestellt. Die Kommunikation zwischen Komponenten C50 und C60 funktioniert wie vorgesehen.

11 Integrationstests (2025-07-09)

Art des Tests: Integrationstest

Ausgeführte Testfälle: T550

Beteiligte Tester: Luan Hyseni

Abgedeckte Komponenten: <C20 + C70> (Konfigurator + Validierung)

11.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendete Versionen: Python 3.13, Tkinter, PyBullet, SkyOrchestrator (Stand: 09.07.2025)

11.2 Testprotokoll

Testfall	T550 – Integrationstest: Konfigurator und Validierung
Tester	Luan Hyseni
Eingaben	1. Formationstyp: Eigener Text 2. Drohnenanzahl: leer gelassen 3. Simulation starten geklickt
Soll - Reaktion	Die Anwendung erkennt die fehlende Eingabe Es wird ein Hinweis oder Fehlerdialog angezeigt Die Simulation wird nicht gestartet Die Anwendung bleibt stabil
Ist – Reaktion	Die GUI zeigte eine Fehlermeldung wegen fehlender Drohnenanzahl Simulation wurde korrekt blockiert Keine Fehlermeldungen in Konsole, saubere Behandlung
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Keine erforderlich

11.3 Zusammenfassung

Der Integrationstest T550 wurde erfolgreich durchgeführt. Fehlerhafte Eingaben im Konfigurator wurden korrekt erkannt und durch das Validierungsmodul abgefangen. Die Komponente C70 verhindert zuverlässig den Start ungültiger Szenen.

12 Integrationstests (2025-07-09)

Art des Tests: Integrationstest

Ausgeführte Testfälle: T560

Beteiligte Tester: Luan Hyseni

Abgedeckte Komponenten: <C20 + C30 + C60 + C90> (Konfigurator, Projektmanagement, Simulation, Visualisierung)

12.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendete Versionen: Python 3.13, Tkinter, PyBullet, SkyOrchestrator (Stand: 09.07.2025)

12.2 Testprotokoll

Testfall	T560 – Integrationstest: Gesamtablauf
Tester	Luan Hyseni
Eingaben	1. Szene erstellt mit Formation: Kreis 2. Drohnenanzahl: 15 3. Lichteffect: Rot (konstant) 4. Projekt gespeichert und Anwendung geschlossen 5. Projekt geladen und Simulation gestartet
Soll - Reaktion	Projekt wird vollständig gespeichert Alle Inhalte werden korrekt wiederhergestellt Drohnen fliegen in Kreisformation mit rotem Licht Simulation läuft stabil ohne Fehler
Ist – Reaktion	Projekt wurde erfolgreich gespeichert und geladen Szene wurde korrekt dargestellt Simulation funktionierte wie erwartet
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine

Nacharbeiten	Keine erforderlich
--------------	--------------------

12.3 Zusammenfassung

Der Integrationstest T560 wurde erfolgreich durchgeführt. Die Szene konnte korrekt erstellt, gespeichert, geladen und simuliert werden. Die Systemkomponenten arbeiteten fehlerfrei zusammen.

13 Integrationstests (2025-07-09)

Art des Tests: Integrationstest

Ausgeführte Testfälle: T561

Beteiligte Tester: Luan Hyseni

Abgedeckte Komponenten: <C20 + C30 + C60 + C90> (Konfigurator, Projektmanagement, Simulation, Visualisierung)

13.1 Testumgebung

Der Test wurde unter Windows 11 auf einem lokalen Testsystem durchgeführt. Verwendete Versionen: Python 3.13, Tkinter, PyBullet, SkyOrchestrator (Stand: 09.07.2025)

13.2 Testprotokoll

Testfall	T561 – Integrationstest: Mehrszenen-Simulation
Tester	Luan Hyseni
Eingaben	1. Zwei Szenen nacheinander erstellt – Szene 1: Formation Kreis, blaues Licht, Dauer 4 Sekunden – Szene 2: Formation Linie, rotes Licht, Dauer 5 Sekunden 2. Projekt gespeichert 3. Anwendung geschlossen und erneut gestartet 4. Projekt geladen 5. Simulation gestartet
Soll - Reaktion	Beide Szenen werden nacheinander korrekt dargestellt Farben, Formationen und Zeiten stimmen mit Konfiguration überein Drohnen verhalten sich in der Timeline logisch Keine Fehler oder Unterbrechungen während des Ablaufs

Ist – Reaktion	Beide Szenen wurden vollständig geladen Simulation lief flüssig von Szene 1 zu Szene 2 Formationen und Farben wurden korrekt angezeigt Keine Fehlermeldungen in Konsole oder GUI
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Keine
Nacharbeiten	Keine erforderlich

13.3 Zusammenfassung

Der Integrationstest T561 wurde erfolgreich durchgeführt. Zwei Szenen konnten im selben Projekt definiert, gespeichert, geladen und in der Simulation nacheinander abgespielt werden. Der vollständige Ablauf funktionierte wie vorgesehen, die Systemkomponenten arbeiteten korrekt zusammen.

14 Komponententests (2025-07-05)

Art des Tests: Komponententest (Unit-Test)

Ausgeführte Testfälle: **T610**

Beteiligte Tester: Lucas Neidahl

Abgedeckte Komponenten: <**C30**> (Projektmanagement-Modul)

14.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt.

14.2 Testprotokoll

Testfall	<i>T610 – Projektmanagement: Projekt speichern und laden (Unit-Test)</i>
Tester	Lucas Neidahl
Eingaben	1. Erstellung einer Test-Datenstruktur, die ein Projekt mit mehreren Szenen simuliert. 2. Aufruf der Funktion <code>save_project()</code> mit der Test-Datenstruktur und einem temporären Dateinamen (<code>test_projekt.json</code>). 3. Aufruf der Funktion <code>load_project()</code> mit demselben Dateinamen.
Soll - Reaktion	Die Speicherfunktion erstellt eine Datei <code>test_projekt.json</code> ohne Fehler. Die Ladefunktion liest diese Datei ebenfalls fehlerfrei ein. Die Datenstruktur, die von der Ladefunktion zurückgegeben wird, muss exakt mit der ursprünglichen, gespeicherten Datenstruktur übereinstimmen.

Ist – Reaktion	Die Projekt-Datenstruktur wurde erfolgreich in die Datei <code>test_projekt.json</code> geschrieben. Anschließend wurde die Datei ohne Fehler wieder eingelesen. Ein Vergleich zwischen der ursprünglichen und der geladenen Datenstruktur bestätigte, dass alle Daten vollständig und unverändert wiederhergestellt wurden.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

14.3 Zusammenfassung

Der Komponententest **T610** wurde erfolgreich durchgeführt. Die Kernfunktionen des Projektmanagement-Moduls zum Speichern und Laden von Projekten arbeiten wie spezifiziert. Die Datenintegrität bleibt während des gesamten Zyklus erhalten, was sicherstellt, dass ein gespeicherter Projektzustand exakt wiederhergestellt werden kann. Die Komponente arbeitet zuverlässig.

15 Komponententests (2025-07-05)

Art des Tests: Komponententest (Unit-Test)

Ausgeführte Testfälle: **T611**

Beteiligte Tester: Lucas Neidahl

Abgedeckte Komponenten: <**C30**> (Projektmanagement-Modul)

15.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt.

15.2 Testprotokoll

Testfall	T611 – Projektmanagement: Projekt löschen (Unit-Test)
Tester	Lucas Neidahl
Eingaben	1. Programmatische Erstellung einer temporären Projektdatei zu Testzwecken, z.B. <code>projekt_zum_loeschen.json</code> . 2. Aufruf der Funktion zum Löschen des Projekts über ein Testskript, das den Dateinamen als Parameter übergibt.
Soll - Reaktion	Die Löschfunktion wird ohne Fehler oder Exceptions ausgeführt. Die angegebene Datei (<code>projekt_zum_loeschen.json</code>) wird permanent vom Dateisystem entfernt. Eine anschließende Prüfung, ob die Datei noch existiert, wird die Datei nicht mehr finden.
Ist – Reaktion	Die Funktion zum Löschen wurde erfolgreich und ohne Fehler aufgerufen. Nach der Ausführung war die temporäre Projektdatei nicht mehr im Verzeichnis vorhanden. Dies wurde durch eine Überprüfung im Testskript bestätigt.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.

Nacharbeiten	Nacharbeiten waren nicht erforderlich.
---------------------	--

15.3 Zusammenfassung

Der Komponententest für den Testfall **T611** wurde erfolgreich abgeschlossen. Die Funktionalität zum Löschen von Projektdateien innerhalb des Projektmanagement-Moduls arbeitet zuverlässig und wie spezifiziert. Die Komponente stellt die sichere Entfernung von nicht mehr benötigten Projektdateien vom Dateisystem sicher.

16 Testdurchführung (2025-07-08)

Art des Tests: Komponententest (Unit-Test)

Ausgeführte Testfälle: **T620**

Beteiligte Tester: Marvin Voigt

Abgedeckte Komponenten: <C60> (Simulation)

16.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 10 (64 Bit), Intel Core i7-2600K, 8 GB RAM, NVIDIA GeForce GTX 970, Python-Version 3.13, pytest, SkyOrchestrator (Stand 08.07.2025), PyBullet wurde im Test gemockt.

16.2 Testprotokoll

Testfall	<i>T620 – Szenenübergabe an die Simulation (Unit-Test)</i>
Tester	<i>Marvin Voigt</i>
Eingaben	Eingaben werden als Dictionary an <code>run_simulation()</code> übergeben. Formationen die getestet werden <code>#heart</code> , <code>#square</code> Beispiel Dictionary: Formation: <code>#heart</code> Anzahl Drohnen: 20 Drohnenmodell: <code>quadrotor</code> Hintergrund: <code>#standard</code>
Soll - Reaktion	Die Funktion <code>run_simulation()</code> sollte alle Eingaben korrekt verarbeiten und die Drohnen entsprechend der Formation „ <code>#heart</code> “ laden. Die Simulationsfunktionen von PyBullet werden im Test gemockt; es werden keine echten Fenster geöffnet. Die Funktion sollte ohne Fehler durchlaufen und alle erwarteten Methoden (z.B. <code>p.loadURDF()</code>) aufrufen.

Ist – Reaktion	Der Test wurde mit <code>pytest</code> mittels folgendem Kommando ausgeführt: <pre>python -m pytest test/unit/test_run_simulation.py</pre> Das Testergebnis lautete: <pre>==== 2 passed in 1.20s ====</pre> Die Funktion <code>run_simulation()</code> lief fehlerfrei durch, alle Assertions wurden erfüllt, und die erwarteten Methodenaufrufe wurden durchgeführt.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

16.3 Zusammenfassung

Der Unit-Testfall T620, der die korrekte Übergabe und Verarbeitung der Szenenparameter im Simulationsmodul prüft, wurde erfolgreich durchgeführt. Die Komponente <C60> (Simulation) verarbeitete sämtliche übergebenen Eingaben wie Formation, Drohnenanzahl, Hintergrund und Drohnenmodell fehlerfrei. Durch das Mocken aller externen Abhängigkeiten konnte sichergestellt werden, dass die relevanten Methodenaufrufe im Simulationsablauf wie erwartet erfolgten. Die Funktionalität wurde ohne Fehler und mit allen vorgesehenen Kontrollmechanismen bestätigt. Die Anforderungen an die Szenenübergabe und Initialisierung der Simulation gelten damit als erfüllt und stabil umgesetzt.

17 Testdurchführung (2025-07-08)

Art des Tests: Komponententest (Unit-Test)

Ausgeführte Testfälle: **T630**

Beteiligte Tester: Lucas Neidahl, Marvin Voigt

Abgedeckte Komponenten: <**C10**> (GUI)

17.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt. Verwendet wurde folgende Testumgebung: Windows 10 (64 Bit), Intel Core i7-2600K, 8 GB RAM, NVIDIA GeForce GTX 970, Python-Version 3.13, pytest, SkyOrchestrator (Stand 08.07.2025), PyBullet wurde im Test gemockt.

17.2 Testprotokoll

Testfall	T630 – Validierung: Eingabefehler erkennen (Unit-Test)
Tester	Lucas Neidahl, Marvin Voigt
Eingaben	In der GUI werden unter „Eigener Text/Formation“ Felder erzeugt und gezielt mit ungültigen Werten befüllt. Anschließend wird auf „Simulation starten“ geklickt. Fall 1: Erstes Textfeld leerlassen. Fall 2: Nicht-numerischer Zeitpunkt (z.B. abc). Fall 3: Negative Anzahl Drohnen (z.B. -10). Fall 4: Im Feld „Anzahl Drohnen“ wird ein ungültiger Wert eingegeben, z.B. abc. Fall 5: Die Formation Textwechsel wird ausgewählt und beim Feld Anzahl Textwechsel wird ein ungültiges non-Integer Zeichen wie 'a' definiert.

Soll - Reaktion	Start der Simulation wird in <code>on_submit()</code> verhindert. Fall 1: „Fehler: Textfeld 1 darf nicht leer sein.“ Fall 2: „Fehler: Ungültiger Zeitpunkt für Text 1.“ Fall 3: „Fehler: Anzahl der Drohnen muss eine positive Zahl sein.“ Fall 4: Fehler: Ungültige Anzahl Drohnen. Bitte eine Zahl eingeben. Fall 5: Fehler: Bitte mindestens einen Text definieren. Die Validierungslogik sollte alle Eingabefehler zuverlässig erkennen und entsprechende Fehlermeldungen ausgeben.
Ist – Reaktion	Simulation wurde blockiert; Fehlermeldungen wurden in roter Schrift im <code>error_label</code> angezeigt; Anwendung blieb stabil. Alle Testfälle bestanden erfolgreich.
Ergebnis	Bestanden
Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

17.3 Zusammenfassung

Der Komponententest **T630** zur Überprüfung der Validierungslogik war erfolgreich. Die Prüfroutinen erkennen zuverlässig Benutzereingabefehler, verhindern fehlerhafte Ausführungen und geben klare Rückmeldungen. Die Funktionalität entspricht den Anforderungen.

18 Testdurchführung (2025-07-08)

Art des Tests: Komponententest (Unit-Test)

Ausgeführte Testfälle: T641

Beteiligte Tester: Lucas Neidahl

Abgedeckte Komponenten: <C10> (GUI)

18.1 Testumgebung

Der Test wurde unter Windows 10 auf einem lokalen Testsystem durchgeführt.

18.2 Testprotokoll

Testfall	T641 – GUI: dynamische UI-Anpassung
Tester	Lucas Neidahl
Eingaben	1. Dynamische UI-Anpassung: a) Start der Anwendung. b) Im Dropdown-Menü „Typ“ wird die Option „#image“ ausgewählt. c) Anschließend wird im selben Dropdown die Option „Eigener Text/Formation“ ausgewählt.
Soll - Reaktion	1. Dynamische UI-Anpassung: zu b): Der Frame „Bild-Upload“ wird sichtbar. Der Frame „Textwechsel definieren“ bleibt verborgen. zu c): Der Frame „Bild-Upload“ wird verborgen und der Frame „Textwechsel definieren“ wird sichtbar. Die GUI passt sich flüssig an die Auswahl an.
Ist – Reaktion	Die Benutzeroberfläche hat sich bei jeder Auswahl im Dropdown-Menü wie erwartet dynamisch angepasst. Die korrekten Eingabebereiche wurden fehlerfrei ein- und ausgeblendet.
Ergebnis	Bestanden

Unvorhergesehene Ereignisse	Es traten keine unvorhergesehenen Ereignisse auf.
Nacharbeiten	Nacharbeiten waren nicht erforderlich.

18.3 Zusammenfassung

Der Komponententest für den Testfall **T641** wurde erfolgreich durchgeführt. Die getestete GUI-Komponente zeigte das erwartete dynamische Verhalten bei der Anpassung der Benutzeroberfläche je nach Benutzerauswahl. Die Funktionalität entspricht den Anforderungen und trägt maßgeblich zur Benutzerfreundlichkeit und Fehlervermeidung bei.